# Evidence That Can Fail: An AI Coding Agent as Primary Developer of IEC 62304 Class C Medical-Imaging Software — A Single-Case Study of Verification, Review and Approval

Nicolás Bonilla Vargas[1,2,3*]

[1*]Universidad Nacional de Colombia, Bogotá, Colombia.
[2]SRH University München, Munich, Germany.
[3]Daita AI.

Corresponding author(s). E-mail(s): ngbonillav@unal.edu.co;

**Abstract**

**Context.** AI coding agents now write substantial parts of production software, but there is no empirical account of an agent acting as the primary developer of safety-critical, regulated software, where the lifecycle records — requirements, risk controls, verification evidence, problem reports, corrective and preventive actions (CAPAs), approvals — are themselves part of the regulated product.
**Objective.** To describe, for one case, what an AI coding agent produced, which verification and governance failures occurred, which mechanisms detected them, and how the boundary between the agent's work and the human owner's approval was operated and evidenced.
**Method.** Retrospective, exploratory single-case study, following the guidelines of Runeson and Höst, of MSTool-AI: investigational brain-MRI software for multiple sclerosis, developed under an IEC 62304 Class C lifecycle by one human owner (the author), who holds every approval, with Claude Code as the agent. Data: the git history of `main` (595 commits, December 2025 to 31 October 2026), 75 pull requests and their descriptions, 16 CAPA records, 22 problem reports, a risk-control-to-test manifest, 47 negative-control (mutation) record files and the 16 scripts that produce most of them, the CI history and the GitHub commit-verification status of the sign-off commits. Every number is re-derived by published scripts from pinned commits; every coded quotation is checked verbatim.

**Results.** The agent's trailer is on 525 of 595 commits (88.2 %; a lower bound), on 99.4 % of added lines and on all 38 commits that touched a Class C module. A verification summary drafted by the agent recorded 21 of 22 risk controls as verified; adversarial re-verification found 4. In the October analysis, 33 verification-failure episodes were coded; 19 escaped to `main` and persisted a median of 91 days. Mutation-based negative controls became the verification of record for risk controls: the last release ran 873 mutations across 15 scripts (868 detected, 5 declared equivalent), yet a control with seven passing mutation vectors had never executed, a re-anchored mutation went blind on a merged tree, and none of the 100 committed record versions shows an unexpected survivor — survivors were recorded only in prose. Review rounds, mostly by separate agent sessions briefed to refute, reported 189 findings in 29 rounds on nine pull requests (2 blockers, 23 major); a clinical-safety defect (lesions not drawn while a radiologist's confirmation was accepted) was found only by a browser walk-through recorded for marketing footage, and its first fix was found incomplete by the next review. All 31 approval lines present in October were written by the agent; after a CAPA, approvals were recorded in GitHub-verified commits made from the owner's web session (279 lines), but the mechanical replacement of a placeholder also rewrote at least 15 sentences that were not approval fields — including the development plan's instruction to the agent — and marked as approved four documents of a change the pull request did not contain. Incidents outside the verification regime — test runs writing 62 accounts into the production user store through ambient credentials, open self-registration exposing the patient list, typed passwords in logs — were found incidentally — while implementing other changes, during a browser capture for marketing footage, in review — and none by CI; across all 38 records, browser captures and the owner's research studies opened half.
**Conclusions.** The agent's characteristic failure was evidence that looked complete but could not fail. What caught it was evidence that could: checks at the locus of effect, mutations that restore the defect as found, real browsers, and review by sessions briefed to refute. The approval boundary became largely attributable once the owner's act moved out of the agent's session — fully so only when the agent stops holding the owner's credentials — and even then the semantics of the records still depended on the agent's wording. We derive implications for standards, auditors and practitioners.

**Keywords:** AI coding agents; LLM-based software engineering; IEC 62304; medical device software; safety-critical software; mutation testing; negative controls; risk management; code review; software process; regulatory compliance; case study.

# 1 Introduction

AI coding agents — language-model systems that read a repository, edit files, run commands and commit — have moved from autocompletion (Chen et al. 2021) to resolving repository-level tasks (Jimenez et al. 2024; Yang et al. 2024; Wang et al. 2025) and to opening pull requests in open source at scale (Li et al. 2025; Watanabe et al. 2026). Their failures are now studied empirically: rejected agent pull requests (Ehsani et al. 2026), misalignment between agents and their users (Tang et al. 2026),

hallucinated code and packages (Liu et al. 2026a; Spracklen et al. 2025), insecure suggestions (Pearce et al. 2022; Perry et al. 2023), and productivity effects on experienced developers (Becker et al. 2025). Surveys map the field (Fan et al. 2023; Hou et al. 2024; Liu et al. 2026b).

None of these settings is a regulated medical device. In a device, the lifecycle records are not documentation *about* the product: under IEC 62304 (International Electrotechnical Commission 2015) and ISO 14971 (International Organization for Standardization 2019) they are the objective evidence that risk controls exist and work, and under ISO 13485 (International Organization for Standardization 2016) approvals and design reviews must be attributable to people. If the agent writes both the code and the evidence, the question is no longer only whether the code is right, but whether the evidence *can fail* when the code is wrong, and whether an approval can be told apart from a sentence the agent wrote.

This paper reports one case: MSTool-AI, investigational brain-MRI software for multiple sclerosis, developed under an IEC 62304 **Class C** lifecycle — the class for software whose failure can contribute to death or serious injury — by a single human owner and an AI coding agent (Claude Code). The agent wrote nearly all of the code and nearly all of the lifecycle records. The owner held every approval. Over eleven months the project accumulated an unusually dense record of its own failures: a withdrawn verification summary, 16 CAPAs, 22 problem reports, adversarial review reports and mutation records. We mine that record, the git history and the GitHub metadata.

A first analysis of this case, pinned at 17 October 2026, was circulated as a draft. Since then the project changed its verification and governance regime substantially: mutation-based negative controls became the verification of record for every risk control and were re-run at each release; every change went through several rounds of independent review by separate agent sessions; and a CAPA opened *from that draft* replaced agent-written approvals with owner-executed sign-off commits. This manuscript extends the analysis to 31 October 2026 and adds the incidents of the last two weeks.

**Contributions.**

1. To our knowledge, the first documented case of an AI coding agent as primary developer of software under a formal IEC 62304 Class C lifecycle, including its risk-management and quality records (§2.5 positions it against the closest work).
2. A quantified, reproducible account of verification failures, their classes, detection channels and persistence (33 coded episodes), and of how the verification regime evolved in response.
3. An assessment of mutation-based negative controls as the verification of risk controls at release scale (873 mutations per release), including three ways in which they failed.
4. Counts and qualitative analysis of 189 findings from 29 independent review rounds, and of what only a real browser found.
5. An analysis of approval provenance: from 31 agent-written approval lines to owner-executed, GitHub-verified sign-off commits, and the semantic side effects of the mechanism.

6. Implications for standards, auditors, practitioners and tool builders, and open data and scripts.

The paper is written as an honest case study. The agent made serious mistakes, some of which the controls caught and some of which they did not; both are reported. The author is also the subject of the study and its draft was written by the agent under study (§7, §10).

# 2 Background and related work

## 2.1 Medical device software regulation

IEC 62304 (International Electrotechnical Commission 2015) defines software lifecycle processes for medical device software: a development plan that names methods and tools (§5.1, in particular §5.1.4), requirements, architecture, unit and integration verification, release, maintenance, configuration management and problem resolution (§9). Its safety classes A, B and C scale the required rigour; Class C applies where a hazardous situation can result in death or serious injury. ISO 14971 (International Organization for Standardization 2019) requires that each risk control is implemented and its effectiveness verified (§7.2). ISO 13485 (International Organization for Standardization 2016) requires approval of documents before issue (§4.2.4), control of records (§4.2.5) and design reviews whose records name the participants (§7.3.5). IEC 82304-1 (International Electrotechnical Commission 2016) covers health software products, and the IMDRF framework categorises software as a medical device by the significance of the information it provides and the state of the healthcare situation (IMDRF Software as a Medical Device Working Group 2014). In the EU, software with a medical purpose is regulated under the Medical Device Regulation (European Parliament and Council of the European Union 2017) with MDCG guidance on qualification and classification (Medical Device Coordination Group 2025); the AI Act (European Parliament and Council of the European Union 2024) adds obligations for AI systems, including AI-enabled devices. In the US, the FDA's guidance on software validation (U.S. Food and Drug Administration 2002), on premarket submissions for device software functions (U.S. Food and Drug Administration 2023), on AI-enabled device software functions (draft) (U.S. Food and Drug Administration 2025) and on predetermined change control plans (U.S. Food and Drug Administration 2024) address the device and its AI functions; the FDA's guidance on computer software assurance (U.S. Food and Drug Administration 2026) addresses production and quality system software. We found no standard or guidance that addresses an AI system *writing* the device software and its records; the regulatory frame treats such a system, at most, as a development tool to be named in the plan (IEC 62304 §5.1.4).

Empirical work on medical device software engineering has studied failure modes from recall data (Wallace and Kuhn 2001; Alemzadeh et al. 2013), the Therac-25 accidents as the canonical software safety failure (Leveson and Turner 1993), trustworthiness (Fu 2011), and the tension between agile or continuous practices and regulatory compliance (McHugh et al. 2012; Heeager and Nielsen 2018; Laukkarinen et al. 2017,

2018; Granlund et al. 2021; Stirbu et al. 2023). None of these studies has an AI agent as developer.

## 2.2 Mutation testing as a negative control

Mutation analysis seeds small faults into a program and asks whether the tests detect them (DeMillo et al. 1978; Jia and Harman 2011; Papadakis et al. 2019). Mutants are, within limits, a valid substitute for real faults (Just et al. 2014); mutation scores correlate with real-fault detection only partly (Papadakis et al. 2018); equivalent mutants — changes that cannot alter behaviour — are a long-standing obstacle (Madeyski et al. 2014). Industrial practice favours targeted mutants surfaced during code review over whole-program scores (Petrović and Ivanković 2018; Petrović et al. 2022). The oracle problem (Barr et al. 2015) reminds that a test can only detect what its oracle observes. The project studied here did not use mutation analysis to score a test suite; it used hand-written, targeted mutations as **negative controls** for individual risk controls: remove or neutralise the control one mutation at a time and require that a bound test fails (§3.3). This is closer to the experimental idea of a negative control than to a mutation score.

## 2.3 Empirical studies of LLM coding assistants and agents

Large datasets of agent-authored pull requests now exist. Li et al. report more than 456,000 pull requests by five agents, accepted less often and simpler than human ones (Li et al. 2025). Watanabe et al. studied 567 pull requests created with Claude Code in 157 projects; 83.8 % were accepted and 54.9 % of those merged without change (Watanabe et al. 2026). Ehsani et al. analysed about 33,000 agent pull requests and found that rejected ones are larger, fail CI more often, and are rejected for socio-technical reasons (Ehsani et al. 2026). Tang et al. coded 20,574 sessions in 1,639 repositories into seven kinds of developer–agent misalignment, including constraint violations and inaccurate self-reports by the agent (Tang et al. 2026). A randomised trial found experienced open-source developers slower with early-2025 AI tools (Becker et al. 2025). Security studies show insecure suggestions (Pearce et al. 2022) and overconfidence of users (Perry et al. 2023). Hallucinations in generated code (Liu et al. 2026a) and hallucinated package names (Spracklen et al. 2025) are documented. These studies sample many projects and short interactions; none follows one regulated product through months of agent-led development, and none can observe how agent-written *verification evidence* behaves.

## 2.4 Agents in safety-critical and healthcare settings

Ferris and Bonilla describe a production healthcare platform built with coding agents and governed by an operator without software-engineering training; supervising agents failed by monitoring proxies, auditing silently and losing checks without notice; the study is qualitative and the system is not a regulated device (Ferris and Bonilla 2026). Davis et al. report a twelve-week first-person case of one engineer with frontier agents (about 420,000 lines of production code) and propose "governance conversion": failures exposed by speed are converted into durable controls (Davis et al. 2026). Wang

**Table 1** Closest related work.

| Work | Setting | Regulated lifecycle (IEC 62304 / ISO 14971) | Quantified verification failures | Approval provenance analysed |
|---|---|---|---|---|
| Ferris & Bonilla 2026 (Ferris and Bonilla 2026) | healthcare platform, agents, non-SE operator | no ("we do not assess compliance") | qualitative | no |
| Davis et al. 2026 (Davis et al. 2026) | one engineer, frontier agents, 12 weeks | no | field notes, not per failure | no |
| Wang et al. 2026 (Wang et al. 2026) | in-house clinical tool, Claude Code | explicitly below device level | none | no |
| Maharjan 2026 (Maharjan 2026) | position paper | discussed | none | discussed |
| Watanabe et al., Ehsani et al., Li et al., Tang et al. | many repositories | no | PR- or session-level | no |
| **This study** | Class C investigational device, one owner + Claude Code, 11 months | yes (records are data) | 33 episodes; 873 mutations per release; 189 review findings | yes (line-level `git blame`, signature verification) |

et al. built an in-house brachytherapy inventory tool with Claude Code in about 13 hours and state that their verification does not reach the level expected for a medical device (Wang et al. 2026); Witztum et al. discuss quality assurance for AI-coded radiotherapy software (Witztum et al. 2026). Maharjan argues, without data, that AI assistants can fit IEC 62304, ISO 13485, ISO 14971 and FDA expectations given traceability, verification and explicit human oversight (Maharjan 2026). Gangopadhyay et al. propose an AI framework for medical-device safety documentation with recorded expert review (Gangopadhyay et al. 2026). Sun and Staron interviewed ten embedded-systems experts on agentic pipelines (Sun and Staron 2026). Studies of LLM code generation for automotive and other safety-critical software evaluate prototypes or snippets (Liu et al. 2024; Kirchner and Knoll 2025; Nouri et al. 2025).

## 2.5 Positioning

Table 1 contrasts this case with the closest work.

The broad claim "nobody has studied coding agents in healthcare" is not safe; the narrow claim — an agent as primary developer of software under a formal Class C lifecycle, with its regulatory records analysed quantitatively — survived searches on 17 and 21 October 2026 and the reference verification of 31 October 2026 (OWNER_REVIEW item B3: re-run before submission).

# 3 The case

## 3.1 Product

MSTool-AI is investigational software for brain MRI in multiple sclerosis: a web viewer (React, TypeScript), a FastAPI backend, lesion segmentation, brain volumetry, MAGNIMS region classification, dissemination-in-space (DIS) support and an AI-drafted report that a clinician must review. Eight backend and frontend modules are designated Class C. The software is **not** CE-marked, not FDA-cleared and not in clinical use; it is deployed for evaluation. No patient data were used in this study.

## 3.2 People, agent and tooling

One human owner (the author) directs the work, holds every approval and is the only human contributor. The agent is Claude Code; commit trailers name eight model strings over the period, from "Claude" (December 2025) through Claude Opus 4.5, 4.6 and 4.8 to Claude Opus 5.5 (September to October 2026) (§5.1). The agent works in the owner's development environment, commits under the owner's git identity and pushes with the owner's GitHub credentials; its only voluntary marker is a `Co-Authored-By` trailer and, in pull-request descriptions, a "Generated with Claude Code" footer. The project instructions (`CLAUDE.md`) require traceability to requirements and hazards, review of Class C changes and owner approval before merge. Since 23 October 2026 the development plan declares the agent as a development tool (SDP-001 v1.1, PR #49, IEC 62304 §5.1.4).

Several agent sessions often run in parallel in separate git worktrees: an implementing session, one or more *independent review* sessions briefed to refute the change, a *coordinating* session that integrates branches into release branches, and sessions that drive a real browser. All are instances of the same agent family; the project's own plan states that they are not independent of the author in the sense of IEC 62304 or ISO 13485 (SDP-001 "Known limitation").

## 3.3 The verification regime as it evolved

Four phases are visible in the records (Figure 1).

1. **Until July 2026: verification by inspection.** Risk-control verification was recorded in prose ("code inspection"), signed "Development Team", with a CI pipeline the agent had built.
2. **From 18 July 2026 (CAPA-001): verification that can contradict the record.** After an adversarial re-verification withdrew a summary that claimed 21 of 22 controls verified, every risk control had to be bound in a machine-checked manifest to named tests, enforced in CI, with a recorded negative control. A negative control that reports "all passed" is inconclusive, never a pass.
3. **From late September 2026: adversarial review and behaviour-level tests.** Separate agent sessions reviewed changes before merge; tests moved to the locus of effect (requests through the mounted application, real headless browsers, store–load round trips); from mid-October every negative-control script had to include a mutation that restores "the defect as found".

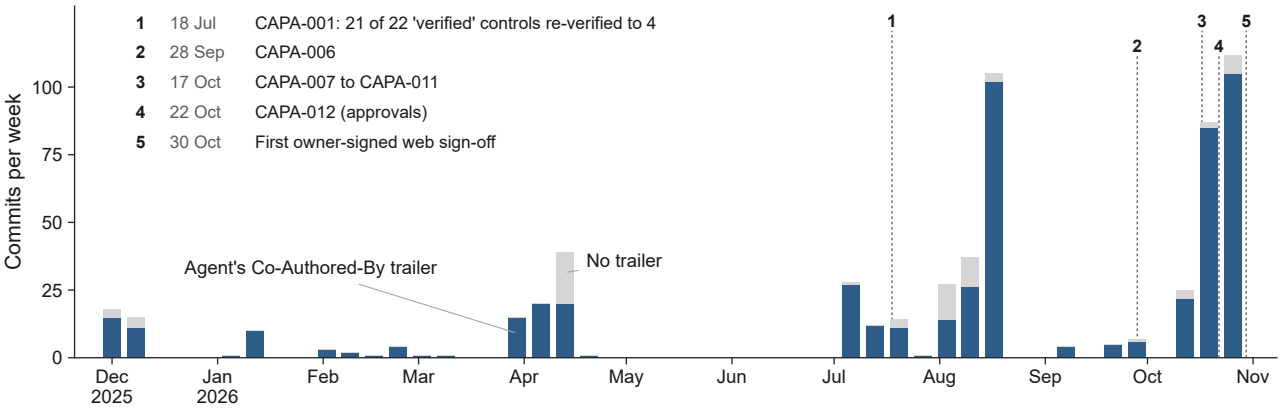



**Fig. 1** Commits on `main` per week, with (blue) and without (grey) the agent's `Co-Authored-By` trailer (525 of 595 commits carry it), and the five numbered events that changed the regime, listed in the figure.

4. **From 22 October 2026: release integration and owner sign-off.** Changes were integrated into release branches (PRs #64, #69, #72), every negative-control script was re-run on the integrated tree in a throwaway worktree, tests were written first and run on the pre-fix base ("tests on base"), every backend run was isolated from cloud services (§5.6), and approvals were recorded by the owner (§5.5).

### 3.4 The approval regime as it evolved

Until 21 October 2026 the owner approved changes in chat; the agent transcribed the approval into the controlled documents ("N. G. Bonilla Vargas — approved … (PR #N)") and the owner merged on GitHub. The October analysis showed that all 31 such lines had been written by the agent (§5.5). CAPA-012, opened on 22 October from that analysis, required that an approval be recorded only "by an action of the owner that the agent cannot perform": a sign-off commit made by the owner, without the agent's trailer, signed with a key the agent cannot use and shown as verified by GitHub, which replaces the placeholder `PENDING owner approval` in the documents of the pull request it names. From 30 October the owner made such commits in GitHub's web interface (github.dev, according to the description of PR #75), where GitHub signs commits with its own key and marks them verified (GitHub 2026).

## 4 Study design

### 4.1 Research questions

The study is exploratory and descriptive (Runeson and Höst 2009; Runeson et al. 2012; Easterbrook et al. 2008). We ask:

- **RQ1 — Contribution.** How much of the code and of the lifecycle records did the agent produce?
- **RQ2 — Verification failures.** Which verification failures occurred, of which kinds; what introduced and what detected each; how long did each persist?
- **RQ3 — Negative controls.** How did mutation-based negative controls work as the verification of risk controls, and where did they fall short?
- **RQ4 — Independent review.** What did independent review rounds find, by severity and channel, and what did they miss?
- **RQ5 — Approval boundary.** How was the boundary between the agent's work and the owner's approval operated and evidenced, and what did the change of mechanism achieve?
- **RQ6 — Incidents.** Which incidents occurred outside the verification regime, how were they found, and how were they contained?

Process data (pull requests, records, risk controls, tests, CI) are reported as context (§5.0).

## 4.2 Case and units of analysis

A holistic single case with embedded units of analysis (Yin 2018): commits, pull requests, review rounds, review findings, CAPA and problem records, risk controls and mutation records. The case was selected because it is, to our knowledge, unique (a revelatory case in Yin's sense): Flyvbjerg's argument that a single case can support analytical generalisation applies (Flyvbjerg 2006). The period is 7 December 2025 (first commit) to 31 October 2026 (merge of PR #75, commit `8805882`).

## 4.3 Data sources and triangulation

Two weaknesses of the triangulation are structural. First, most textual sources (D2, D3, D6, the PR descriptions in D7) were written by the agent; only D1, the manifest checks, the CI results, the GitHub verification status and the record files produced by scripts are machine evidence. Second, the reviewers' own reports are not in the repository; what is known of each review round comes from the agent's summary of it in a PR description or record.

## 4.4 Procedures

All data were read with read-only commands on pinned commits (`git log`, `show`, `ls-tree`, `grep`, `blame`, `rev-list`) and with read-only `gh` queries, whose output is committed as a snapshot. Two tokens that name an unrelated product of the owner were redacted from the snapshot by hash. No production system, cloud service or patient data was accessed. The scripts are in `research/paper_h/`: `run_all.py` (October analysis, pinned at `6209131`), `manuscript_data.py` (this update, pinned at `8805882`), `snapshot_github_full.py` (snapshot) and `docs/research/paper-h/manuscript/figures/make_figures.py` (figures). `manuscript_data.py --check` fails if any coded quotation, stated total or record count does not match its source.

**Table 2** Data sources.

| ID | Source | Nature | Access |
|---|---|---|---|
| D1 | git history of `main` @ `8805882` (595 commits) and @ `6209131` (October analysis) | primary, machine-generated (dates, trailers, committer, blame) | local, read-only |
| D2 | Controlled documents: SDP-001, SRS-001, RMF-001, TM-001 and others | records written by the agent, approved by the owner | at pinned SHAs |
| D3 | CAPA-001..016, PRB-001..031, CAPA register | records written by the agent | at pinned SHAs |
| D4 | Risk-control manifest (`rc_test_manifest.json`) across first-parent history | machine-checked record | git history |
| D5 | 47 negative-control record files (21 per-change, 26 per-release); 100 distinct committed versions; 16 negative-control and 6 tests-on-base scripts | machine-generated by agent-written scripts | git history |
| D6 | Release integration notes 2026-10-25 and 2026-10-30 | records written by the agent | at `8805882` |
| D7 | GitHub: 75 PR descriptions, reviews, comments, merges, 291 CI runs, checks on PR heads, branch protection, commit verification (snapshot 2026-10-31T14:51Z) | platform metadata + agent-written text | read-only `gh`, committed snapshot (redacted) |
| D8 | Owner's notes on the permission classifier (two refusals, 17–21 October) | owner's notes, outside the repository | cited via CAPA-012 Finding 4; not reproducible |
| D9 | October analysis: coding sheet of 33 episodes, 81 verbatim quotations, results | derived, scripts published | `research/paper_h/` |
| D10 | Review-finding coding (189 findings) and record-channel coding (38 records) | derived, single coder (the agent), quotes checked | `research/paper_h/data/` |

## 4.5 Coding

**Verification-failure episodes (RQ2; October protocol).** A verification-failure episode is a documented instance in which recorded evidence asserted a property the system did not have, an executed check could not observe the failure it targeted, a negative control was invalid or blind, or an agent-drafted record stated a falsehood. Episodes were coded into seven a-priori classes — **P** prose or self-attested; **N** never executed or could never fail; **S** wrong locus; **I** environment could not observe; **M** negative control invalid or blind; **R** false statement in an agent-drafted record; **C** clinical or labelling content not traced to its source — and into detection channels (adversarial review AR, behaviour probe BP, systematic sweep SA, CI, incidental discovery while implementing IM, external research EX, owner OW, and the post-hoc self-check SC). For each episode the introducing and detecting commits were identified and the stage checked against git ancestry: *escaped* (the defective evidence reached `main` before detection) or *contained*. The protocol was committed before the analysis scripts (`08e952e`).

**Review findings (RQ4).** Each review round reported in a PR description or record for PRs #64–#75 was coded: round, channel, verdict, and each finding with the severity *as labelled by the source* (blocker, major, minor, nit; unlabelled when the source gives none), with a verbatim quotation. A finding repeated in a PR description

**Table 3** Triangulation of the main claims. $P$ = primary evidence, $C$ = corroborating evidence.

| Claim | D1 git | D2–D3 records | D4–D5 manifest, mutations | D6 notes | D7 GitHub | D8 notes |
|---|---|---|---|---|---|---|
| Agent wrote most code and records (RQ1) | P (trailers, lines, blame) | C | | | C (footer on 75/75 PRs) | |
| 21/22 → 4/22 verified (RQ2) | C (authoring commits) | P (both summaries) | C (manifest after CAPA-001) | | | |
| Escape and persistence of failures (RQ2) | P (ancestry, dates) | P (CAPA texts) | C | | | |
| Mutation totals per release (RQ3) | C (record commits) | | P (record files) | P (stated totals) | | |
| Survivors only in prose (RQ3) | P (record history) | | P | P (prose) | | |
| Review findings (RQ4) | | C (PRB/ CAPA review sections) | C (re-run records) | P | P (PR descrip- tions) | |
| No formal review; no branch protection (RQ5) | | C (CAPA-012) | | | P (API) | |
| Approval lines and their writers (RQ5) | P (`git blame`, committer) | P (lines) | | | P (verification status) | |
| Permission classifier refusals (RQ5) | | C (CAPA-012 F4) | | | | P |
| Incidents (RQ6) | C (fix commits) | P (PRB-002, -020, -021) | C (controls) | C | P (PR #44, #70) | |

and a record was counted once; a finding carried open into a later round was not counted again; new identifiers opened by incomplete fixes were counted.

**Detection channel of records (RQ6, context).** Each of the 38 CAPA and problem records was coded by its own *Source* or *Reporter* field into the channels of Figure 7, with a verbatim quotation.

**Sign-off substitutions (RQ5).** Every occurrence of the placeholder removed by a sign-off commit was classified heuristically as an approval placeholder or as one of three non-placeholder uses: the name of the marker inside procedure text or a quotation; the marker conjoined with a review ("PENDING owner approval and code review"); a statement about the state before sign-off.

All codes were assigned by the agent as a single coder. A human second coder is required before submission (§7, OWNER_REVIEW B1).

**Table 4** Process data at `main` @ `8805882` (31 October 2026), with the October analysis (`6209131`, 17 October) for comparison.

| Measure | 17 Oct (`6209131`) | 31 Oct (`8805882`) |
|---|---|---|
| Commits on `main` (agent-trailered) | 388 (327) | 595 (525) |
| Pull requests (merged / open / closed) | 56 (47 / 9 / 0)[a] | 75 (64 / 9 / 2) |
| Formal GitHub reviews on any PR | 0 | 0 |
| CAPA records on `main` (Critical) | 9 (8)[c] | 16 (15) |
| Problem reports (PRB) | 0 | 22 (6 Critical, 11 Major, 5 Minor) |
| Risk controls in the manifest (implemented) | 35 (18) | 88 (68) |
| Backend test functions / frontend test cases declared | 1,056 / 116 | 1,502 / 584 |
| Negative-control scripts / tests-on-base scripts | 4 / 0[b] | 16 / 6 |
| CI workflow runs since 3 April 2026 (success) | — | 291 (218) |
| `main` branch protection | none (HTTP 404) | none (HTTP 404) |

[a] GitHub snapshot of 21 October. [b] scripts on `main`; three more were on open PR branches. [c] CAPA-010 and CAPA-011 existed only on open PR branches (11 records in the October analysis).

### 4.6 Analysis

The analysis is descriptive. Counts, medians and ranges are reported; no inferential statistics are computed, because the units are neither sampled nor independent (mutations are designed; review rounds of one PR depend on each other). Qualitative analysis follows the episodes and incidents as narrated in the records, checked against git.

### 4.7 Protocol and deviations

The October analysis followed a protocol committed before its scripts (`PROTOCOL.md`); its post-hoc additions are listed in `RESULTS.md` §5. This update is **post hoc in its entirety**: RQ3–RQ6 as formulated here, the review and channel codings, the substitution classification and the record-status check were defined after the events. The October episode set (RQ2) was not extended to the period 22–31 October; episodes of that period are reported qualitatively (§5.2.4) rather than coded, to keep the protocol-governed set intact.

## 5 Results

### 5.0 Process overview

Between the two snapshots, 207 commits reached `main` (198 agent-trailered) and 19 pull requests were opened (#57–#75), 13 of them merged, three as release-integration branches (#64, #69, #72). All five new CAPAs and all 22 problem reports were opened between 22 and 31 October (Figure 2). Records reached `main` quickly — median 6 days from opening for CAPAs (range 0–19) and 2 days for problem reports (0–6) — but **none** of the 16 CAPAs is closed and all 22 problem reports remain

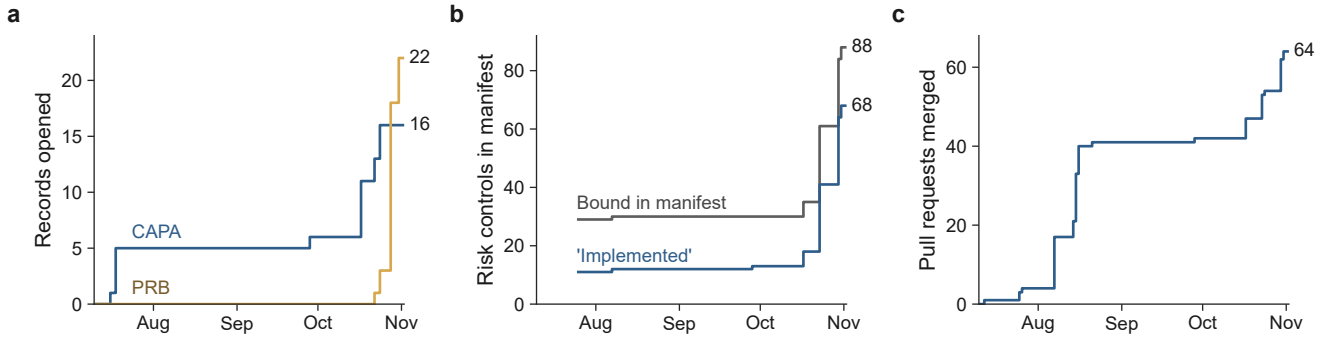



**Fig. 2** Cumulative CAPA and problem records opened (a), risk controls bound in the manifest and those marked 'implemented' (b), and pull requests merged (c), July–October 2026. The values at the line ends are the totals on 31 October.

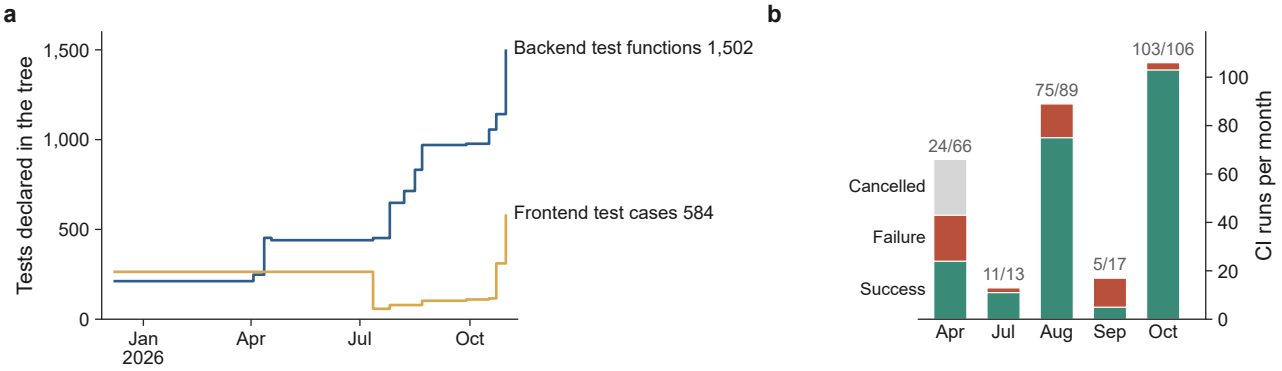



**Fig. 8** (a) Tests declared in the tree, weekly (the July drop in frontend test cases is the removal of test files of deleted modules in a clean-up merge, PR #1, `0c7b08b`). (b) CI workflow runs per month by conclusion, labelled successful/all (291 runs of the seven-job workflow).

"OPEN — DRAFT". The status fields lag behind the history: 25 of the 38 records on `main` still describe their fix as being on a branch, "not merged" or in a draft pull request, although the fix is on `main` (`records.rows[*].status_still_says_not_merged`). Whether the fixes are deployed cannot be determined from the repository (OWNER_REVIEW C5).

The CI workflow (seven jobs: type check, build, frontend tests, backend tests, risk-control verification, dependency vulnerability scan, static analysis) ran 291 times; the last head of each of 71 pull requests with checks was green on all seven jobs. CI is not a merge gate: `main` is not protected and has no rulesets (§5.5). The number of declared tests grew by 42 % (backend) and fivefold (frontend) in two weeks (Figure 8); the release records report 2,309 backend unit tests and 621 frontend tests passing on the final integrated tree, offline. These counts are agent-reported (the record) or proxies (declarations in the tree), not independent measurements of adequacy.

## 5.1 RQ1 — The agent as primary developer

Of 595 commits on `main`, 525 carry the agent's `Co-Authored-By` trailer (88.2 %); of 546 non-merge commits, 513 (94.0 %). Agent-trailered commits added 357,958 lines and untrailered ones 2,100 (agent share 99.4 %; 99.3 % without the agent-trailered initial import of 69,839 lines). All 38 non-merge commits that touched one of the eight Class C modules carry the trailer. Of 163 non-merge commits that touched a controlled document, 159 carry it; the four that do not are the owner's three web sign-off commits and one unsigned local approval commit of 23 October (`8832dae`, §5.5). Every one of the 75 pull-request descriptions ends with the Claude Code footer.

The trailer is a lower bound on the agent's involvement and a self-report: the agent adds it voluntarily, and the repository cannot distinguish an owner commit from an agent commit without it. The 33 untrailered non-merge commits cluster on a few dates (December 2025 deployment fixes, April and August 2026) and, since the October analysis, on the four approval commits. Eight model strings appear in trailers; Claude Opus 5.5 (with and without the "1M context" suffix) wrote 223 commits, all after 27 September 2026.

In short, the agent wrote nearly all code, all tests, all records and all review reports; the owner's direct contributions in the repository are the merges (64 of 64 merged PRs show the owner's account as merger), the three web sign-off commits, and — not visible in the repository — the instructions, decisions and approvals given in chat.

## 5.2 RQ2 — Verification failures, their detection and persistence

### 5.2.1 The withdrawn verification summary

A verification summary created by the agent on 12 April 2026 (`b6be6e7`) recorded "Verified 21 (95.5 %) … Failed 0", signed "Development Team (code inspection)". An adversarial re-verification on 18 July 2026 by a separate agent session, briefed to falsify, found 4 verified, 6 partial and 12 overstated or not implemented (Figure 6a). The summary was withdrawn in place (`213efdf`) and CAPA-001 opened. The re-verification was itself later found incomplete for one control (RC-010, examined on one of three paths).

### 5.2.2 Thirty-three episodes (October analysis)

The October analysis coded 33 verification-failure episodes (53 items) from 11 CAPA records, the two summaries, the manifest, the mutation records, commit messages and review responses; all 81 quoted passages were found verbatim at their pinned refs. The largest class was **I**, environments that could not observe the failure (10 episodes as primary code): a unit-test mock that carried the very field production data lacked, so four routes returned HTTP 500 while the control was "verified"; a safety caveat clipped off-screen while jsdom tests, which have no layout engine, passed; store and load tested separately so that a double transpose went unseen — and cubic fixtures would have hidden it anyway. **S**, the wrong locus (5): a wiring test that read one source file after a refactor had moved ten routes out of it; a patient-authorization

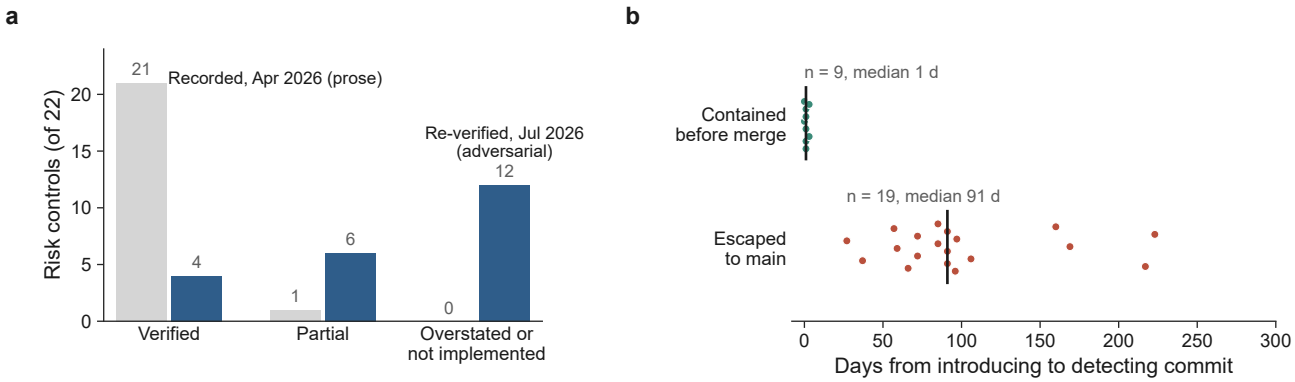



**Fig. 6** (a) The risk-control verification record of April 2026 against the adversarial re-verification of July 2026 (22 risk controls). (b) Days from the introducing to the detecting commit for the 28 dated failure episodes of the October analysis, by stage; each dot is one episode and the vertical bar the median.

dependency written as a plain function returning an un-awaited coroutine, which the web framework never executed, whose tests called the decision function directly and matched the text `Depends(require_patient_access)`. **N** (3): a security test suite never collected because it imported a module that never existed; a syntax check that could not fail because `find -exec` swallows exit codes; a CI path filter that excluded every component test. **P** (1, 12 items): the withdrawn summary. **C** (4): a "40 % CVS rule" presented as McDonald 2024 and an activity threshold attributed to MAGNIMS, both written from memory; labelling issued with thresholds the code had not applied for six weeks; a normative table without provenance. **R** (4): false statements in agent-drafted records, such as "refuses with HTTP 422" when every call returned 500. **M** (6): invalid or blind negative controls (§5.3).

All 28 episodes with an introducing commit were introduced by agent-trailered commits; all 33 detections were recorded by agent-trailered commits. Detection channels: adversarial review by separate agent sessions 15, behaviour probes 6, self-check 4, incidental discovery 3, CI 2, external research 2, route sweep 1, owner 0. The records credit mechanisms, not people; the owner commissioned the audits.

### 5.2.3 Persistence

The 19 escaped episodes persisted a median of 91 days (range 27–223) and were recorded on only four dates — audit bursts on 18 July, 14 August, 28 September and 17 October 2026. Every escaped episode was of class P, N, S, C, R or I; none of class M. The nine contained episodes that reached a commit were introduced on 17–18 October and caught within 0–3 days; five more never reached a commit (Figure 6b). The contrast is right-censored (October failures had little time to escape) and confounded with the change of model and the start of routine pre-merge review.

### 5.2.4 After the October analysis (22–31 October; qualitative)

The period after the October analysis was not coded under the protocol, but its records describe failures of the same shapes, now caught before merge:

- **Cross-change interactions found only at integration.** Merging four reviewed pull requests into release 2026-10-25 produced 151 unit-test failures; in the notes' words, "Each PR's route tests stubbed only their own hooks". At release 2026-10-30 the coordinating session found that, as merged, "every segmentation classified before #71 would have been refused (drawn orange, stored red)", while a client stating colours the viewer never draws would have passed; and that an administrator-created account "would have read as an access grant" in the new audit trail. Each was fixed with tests written first.
- **A clinical-safety defect that review missed and a browser found** (F08, §5.4).
- **A blind negative control on the merged tree** (CAPA-009 O2, §5.3).
- **Records that state an outdated status** (25 of 38, §5.0) and **approval text that changed meaning** (§5.5).
- **Changes without review.** The description of PR #64 states that "**#59 and #62 had no independent review when written**"; the review of the release branch found four problems in them.

## 5.3 RQ3 — Mutation-based negative controls

### 5.3.1 Scale

By 31 October the repository held 16 negative-control scripts and 6 "tests on base" scripts. A negative-control script applies each mutation in a throwaway git worktree, runs the tests bound to the risk control, records which tests failed, restores the tree and checks that the restored suite passes. Most records store SHA-256 hashes of every mutated source and test file and declare in advance which mutations are expected to survive (equivalent mutations) and why; the most recent scripts also refuse to run on uncommitted targets. Mutations are hand-written; each script includes at least one mutation that restores the defect as found.

At release 2026-10-25, 11 records ran 602 mutations (597 detected, 5 declared equivalent). At release 2026-10-30, the first run on the merged tree executed 14 scripts and 772 mutations (767 detected, 5 declared equivalent); after `main` was merged into the release branch, ten scripts were re-run and one added, for a final total of **873 mutations across 15 scripts: 868 detected, 5 declared equivalent, 0 unexpected survivors** (Figure 3a). The record files under `records/releases/` reproduce these totals exactly, and the totals stated in the integration notes match them (`negative_controls.release_2026_10_30_*`). Counting every distinct committed version of every record file reachable from `main`, the history holds 100 recorded runs and 5,925 mutation executions between 28 September and 31 October 2026 (Figure 3b).

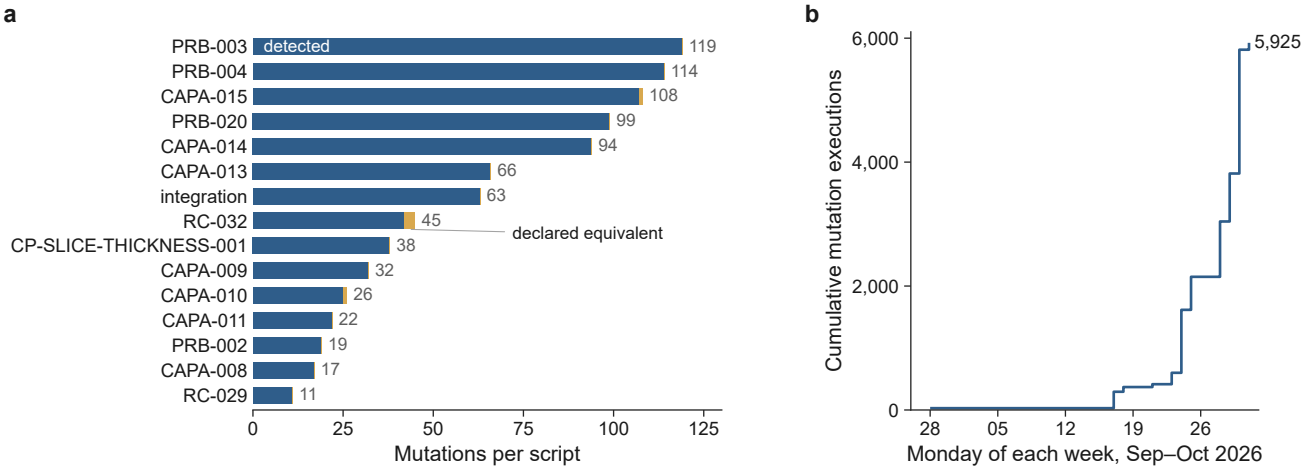



**Fig. 3** (a) Mutations per negative-control script in release 2026-10-30, detected and declared equivalent (873 mutations: 868 detected, 5 declared equivalent; the first run had 772). (b) Cumulative mutation executions in committed record versions (100 recorded runs, 5,925 executions, no unexpected survivor).

### 5.3.2 What negative controls achieved

Negative controls turned verification into evidence that can fail. Under CAPA-001 every implemented risk control names a negative-control result in the manifest (68 of 88 controls on 31 October are "implemented"). Tests written after a defect were run on the pre-fix code: in CAPA-010, 25 of 27 new tests failed on the `main` sources (the other two were expected to pass); in the last release, the failing-first tests for PRB-028..031 failed on the base in 32 of 37 backend and 42 of 49 frontend cases, the rest being declared positive controls. When a merge broke a negative control, the next run said so.

### 5.3.3 Where negative controls fell short

Three failure modes recur.

1. **Locus.** A negative control is only as informative as the place it mutates. RC-026 had seven mutation vectors, all detected, and a control that had never executed: every mutation and every test acted on the decision function, not on the request path (October episode E19, CAPA-009). Three implemented controls on `main` carried an erratum stating that the control did not work in the running system despite passing negative controls.
2. **Drift under merges.** Mutations are anchored to text. When a merge changes the anchored code, the mutation must be re-anchored, and a re-anchor can silently stop removing the control. At release 2026-10-30, "the first run, on `04fd105`, left O2 undetected: the re-anchor after #71 kept the reference image's authorization"; the original two-edit mutation was restored and detected. Earlier, in October, three

hashed files of one record no longer matched the code on `main`; the release records now check every hash against the integrated tree.

3. **Survivorship in the record.** Records are re-run until no unexpected survivor remains, and only then committed. Of the 100 committed record versions, **none** shows an unexpected survivor (`negative_controls.history.runs_with_unexpected_survivor = []`). Every survivor that the process found — the six M-class episodes of the October analysis and CAPA-009 O2 on the merged tree — is known only from prose in the integration notes and CAPA records. A related weakness is detection for the wrong reason: one mutation of the PRB-004 script "was detected through a load error, not an assertion" (`8fd9766`) and had to be rewritten so that the code stayed valid. A pooled detection rate of 99.4 % (868/873) is therefore an artefact of the process, not a measure of test effectiveness; the informative signal is in the prose.

## 5.4 RQ4 — Independent review rounds

### 5.4.1 Counts

No pull request received a formal GitHub review (0 on 75); there were three PR comments, all by the owner's account. Review happened instead in separate agent sessions, reported in PR descriptions and records. For PRs #64–#75 we coded **29 review rounds on nine PRs and 189 findings**: 2 blockers, 23 major, 73 minor, 3 nits and 88 findings without a severity label (Figure 4). PR #69 (the 2026-10-25 release) had six rounds and 46 findings; PR #67 (de-identification of the AI report) five rounds and 29 findings, including a first verdict that it "should not be merged" and a redesign; PR #68 (DIS only on a radiologist-confirmed segmentation) a blocker — "DIS was evaluated on any confirmed mask whatever its labels meant" — and a second review that found three earlier findings "only partly closed". PRs #72, #73 and #75 had no review round (#72 is an integration of reviewed branches; #75 carries only the owner's sign-off commit). The most recent round, on PR #74, found ten minor items and "No blocker and no major issue". By channel, independent agent sessions reported 162 findings, browser capture sessions 25, and a coordinating session 2.

Each finding that led to a change was handled in the same way: a test written first and shown to fail on the reviewed head, the correction, then new or re-anchored negative controls. The records list, per round, the failing-first test runs and their result on the reviewed head (e.g. "backend 3 of 3 and frontend 4 of 5 fail").

### 5.4.2 What only a browser found: F08

Release 2026-10-25 (PR #69) integrated a change requiring that DIS be evaluated only on a segmentation a radiologist confirmed. Its first independent integration review found "nothing reviewed was lost in the integration and no blocker or major issue". On the same commit, a browser capture session — run to record footage of the application for a marketing film — found F08, classified "clinical safety": after "Auto-Classify Regions", "lesions labelled 2–4 (juxtacortical, infratentorial, deep white matter) were not drawn on the 2D image until the study was reopened; a radiologist could confirm without seeing them". The capture session found three more confirmation defects (F09:

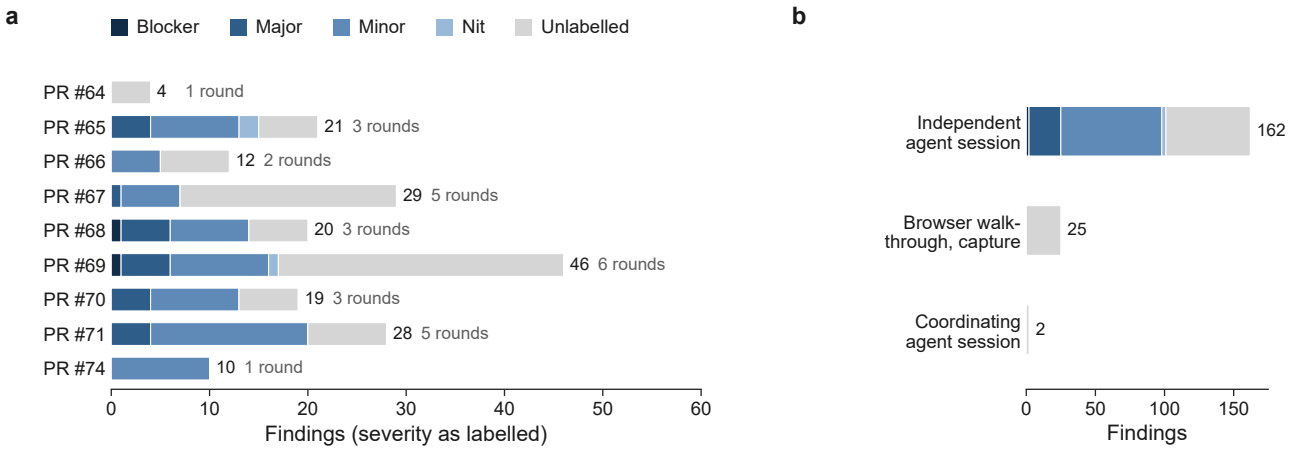



**Fig. 4** Findings of the independent review rounds of PRs #64–#74 (189 findings in 29 rounds), by severity as labelled in the record: (a) per pull request, with the number of rounds; (b) by channel.

small lesion components silently removed from the stored mask before confirmation; F10: a stale "not confirmed" message; F04) that the records say "block the release". The fix was reviewed again: the second independent review found **B1**, "F08 still reachable", through a stale list, and three majors (M1–M3; M3: a label's eye toggle or zero opacity "let a radiologist confirm with lesions hidden"). The final check found two more majors in the confirmation guard (R1: the guard read the 2D canvas settings, not the view on screen; R2: switching the label preset redrew the regions with other names and colours). The last check left two minor items for the owner, later opened as PRB-030 and PRB-031 and fixed in PR #74.

The original defect was therefore found by a walk-through in a real browser, outside any planned verification; its incomplete fix was found by independent review; and the guard that resulted still relies on a check that runs in the browser, which the release records name as the residual risk.

## 5.5 RQ5 — The approval boundary

### 5.5.1 Until 21 October: approvals transcribed by the agent

At `6209131` (17 October), 31 lines in 19 controlled documents recorded the owner's approval; `git blame` attributes all 31 to five agent-trailered commits. Each transcribed an approval the owner had given in chat, which is not part of the repository. The agent committed under the owner's identity and merged with the owner's credentials; three merges on `main` had been made locally in agent-trailered commits. CAPA-012 (Critical) states the consequence: "an approval that cannot be attributed to the approver is not evidence". It also found that `main` "returns HTTP 404 'Branch not protected'", that "the code owner is the pull-request author" — so the required review "can never be given" — and that the project documents described these controls as if they were in place.

### 5.5.2 The permission classifier

The owner's notes (D8, cited by CAPA-012 Finding 4) record that "on two occasions between 2026-10-17 and 2026-10-21 the agent harness's permission classifier refused" to let the agent write the owner's approval line into controlled documents — once after a chat message approving "everything", once after a written authorisation — although the agent had performed the same act five times before. The agent did not change its permissions; the owner was asked to merge on GitHub or to authorise explicitly. CAPA-012 records this as "a behaviour of the tool, not a control of the quality system": it is not described in the plan, the QMS cannot configure or verify it, and the committed permission settings of the project contain one allow rule and no deny rules. Throughout the period, the records leave every production action to the owner: the clean-up of the test accounts written into production (PRB-002), the review of self-registered production accounts and of production logs (PRB-020), deployments and the rotation of an exposed credential (§5.6); they state repeatedly that "production was not contacted". Whether the classifier also refused production actions is not recorded in the repository (OWNER_REVIEW C3).

### 5.5.3 From 22 October: owner-executed sign-off

Figure 5 traces the change. On 23 October, one commit (`8832dae`) without the agent's trailer and without a signature, made locally, replaced the placeholder in 17 files (197 occurrences) for seven pull requests; its provenance is not established by the repository (OWNER_REVIEW C1). From 30 October three sign-off commits were made from the owner's GitHub web session (github.dev, according to PR #75): `5561999` (PR #70, 64 occurrences in 16 files), `c1d0e04` (PR #72, 239 occurrences in 25 files) and `042d50b` (PR #74, 28 occurrences in 8 files). GitHub reports all three as verified (`verification.verified = true`, reason `valid`); they carry no trailer and no Claude Code footer. PR #74 had been merged before its sign-off was recorded; according to PR #75, the owner then signed on the PR branch, and PR #75 brought that one commit to `main`.

At `8805882`, 489 lines in 59 controlled documents record the owner's approval: 279 written by the three verified owner commits, 179 by the unsigned local commit and 31 by the five historical agent-trailered commits, which CAPA-012 keeps unchanged as "approval given by the owner in chat, transcribed by the AI coding agent". No placeholder remains.

### 5.5.4 What the mechanism did not achieve

The sign-off mechanism made the *act* far more attributable than before: the three commits were made through GitHub's web surface, not in the agent's local session, carry neither the agent's trailer nor its footer, and are verified by GitHub. Attribution is still not complete. GitHub's web-flow signature shows that a commit was created through GitHub with the account's credentials, not whose hand made it; the agent's environment holds the owner's GitHub credentials (CAPA-012 Finding 2), and commits created through GitHub's interfaces with those credentials can be signed by GitHub in the same way (GitHub 2026). That the owner made these three commits

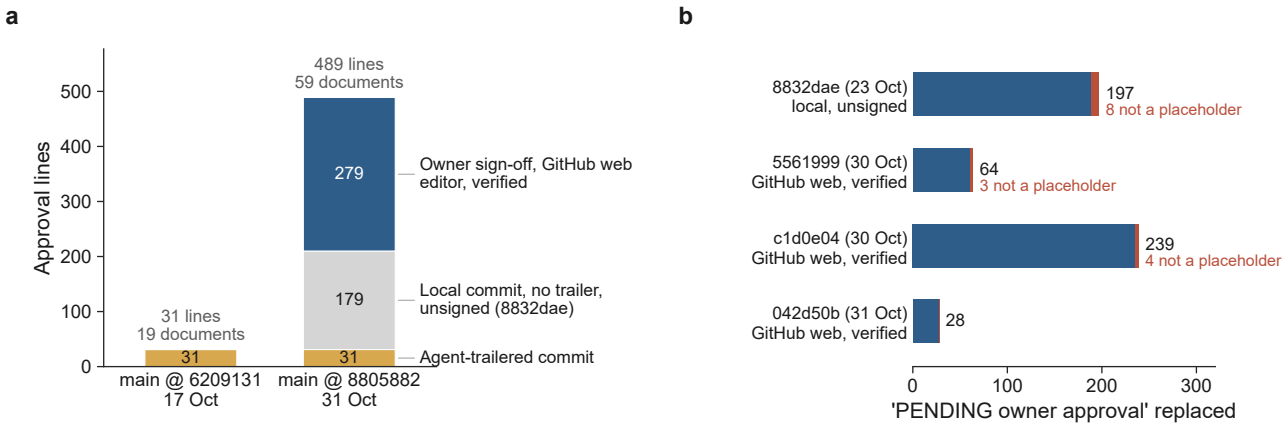



**Fig. 5** (a) Who wrote the lines recording the owner's approval (git blame), on 17 and 31 October. (b) What each sign-off commit replaced: occurrences of 'PENDING owner approval', split into approval placeholders and other text (procedure text, a quotation, '… and code review', a pre-sign-off statement).

rests on the owner's statement, transcribed by the agent in the description of PR #75 (OWNER_REVIEW C1). Only a separate agent identity without the owner's token (CAPA-012 CA-12.3, open) would make the distinction technical. Nor did the mechanism make the *content* of the record the owner's. It replaces every occurrence of a fixed string, and the agent had used that string not only as a placeholder in approval fields but also inside sentences. Of 528 occurrences replaced by the four approval commits, at least 15 (2.8 %, heuristic classification) were not approval placeholders:

- **Procedure text that now contradicts itself.** The development plan's new §7.4, drafted by the agent under CAPA-012, said that "where an approval is pending" the agent writes the placeholder. After the substitution of 23 October it reads: "The agent may draft any document but never writes an approval line … where an approval is pending it writes `approved by N. G. Bonilla Vargas 2026-10-23 (PR #52/#51/#53/#59/#49/#62/#63)`." The same happened to CAPA-012's specification of the CI check (PA-12.1). The description of PR #64 notes that "Some #62 occurrences in SDP-001 §4.4/§7.4 and in CAPA-012 are sentences of the proposed procedure that name the marker, not blocks to sign."
- **A review that did not happen, now recorded as approved.** PRB-020's status read "PENDING owner approval and code review"; it now reads "N. G. Bonilla Vargas — approved 2026-10-30 (PR #70) and code review", although no human code review took place.
- **Statements about the pre-sign-off state.** Integration notes that said "Every approval field in every document still reads 'PENDING owner approval'" now say that every field "still reads" the approval line.
- **Approval outside the pull request's scope.** The sign-off of PR #70 also replaced placeholders in four documents of another change (the in-browser segmentation, CP-FLAMES-BROWSER-001), which PR #70 did not touch. Its change

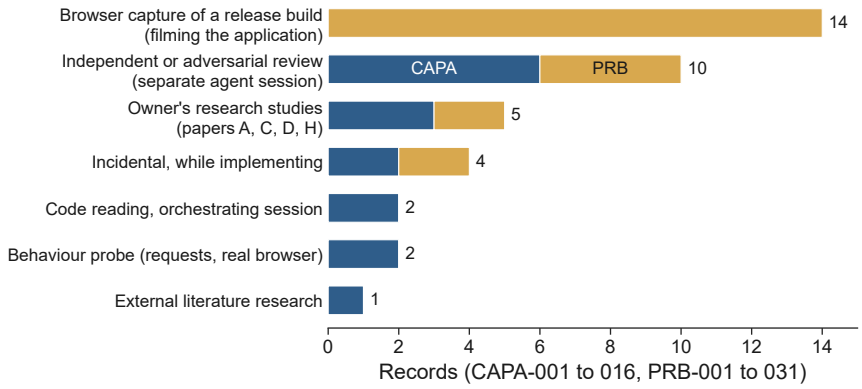



**Fig. 7** How the 38 CAPA and problem records were opened (coded from each record's Source or Reporter field): CAPAs blue, problem reports ochre.

proposal now reads "**Draft records written, NOT approved.** … with every approval field 'N. G. Bonilla Vargas — approved 2026-10-30 (PR #70)'".

In addition, no pull request has a formal review, `main` is still unprotected on 31 October, the agent still acts under the owner's identity (CAPA-012 CA-12.3 is open), and an independent human reviewer for Class C changes has not been appointed (CA-12.5 open).

## 5.6 RQ6 — Incidents outside the verification regime

Figure 7 shows how the 38 CAPA and problem records were opened. The largest single channel was the browser capture of a release build (14 records, all problem reports, all from captures made on 28 October); independent or adversarial review opened 10; the owner's research studies on the same code base — papers A, C, D and this study — opened 5 (including CAPA-012); incidental discovery while implementing opened 4; behaviour probes 2; code reading in an orchestrating session 2; external literature research 1.

Four incidents illustrate failures that the verification regime could not see.

**PRB-002 — tests wrote to production through ambient credentials.** On a computer with cloud application-default credentials, nothing in the backend test configuration hid them; the application created its user store at import and "prefers Firestore when a client is available", defaulting to the production project. Local test runs therefore wrote **62 distinct test accounts** (112 writes) into the production authentication collection between 12 July and 24 October 2026 (the oldest surviving log; earlier runs cannot be excluded). No existing record was changed, the accounts are encrypted with a test key and cannot log in to production, and no clinical data was involved. "The CI pipeline has no cloud credentials, so CI never showed the problem." The agent found it incidentally while developing CAPA-015, and the scope was established from local logs only. The fix (PR #66) is a guard of five independent layers set before any application import: forced non-existent credentials, an empty

cloud-SDK configuration, a fake project with external keys removed, a credential function that always raises, and sockets restricted to loopback with proxies disabled — verified by 19 tests and 19 negative-control mutations. The ambient credentials also carried the quota project of an unrelated product of the owner; no data of that product was read or written. Clean-up of the production records is an owner action (OWNER_REVIEW C2).

**PRB-020/021 — open self-registration exposed the patient list.** A browser capture of the release candidate on 28 October — the same session run for marketing footage — and "a marketing study that named it the largest EU MDR risk" found that anyone could create an account (PRB-020). While reproducing it, the agent found that every signed-in user, of any role, could list the whole patient registry (PRB-021): name, record number, birth date and sex. The record routes had been scoped by CAPA-002 and CAPA-009; the list and search routes "name none" and had never been. Together, anyone on the internet could read the registry. "Whether patient data were read by self-registered accounts in production is unknown"; production was not contacted. The fix (PR #70, merged and signed on 30 October) turns self-registration off and makes it impossible in production, and scopes the list to the user's care-team assignments. The production review and the data-protection assessment are owner actions (OWNER_REVIEW C4).

**Passwords in logs and in the repository.** While testing a finding of the independent review of PR #70, the agent found that "every validation error, the typed password included, was written to the log with its `input`", and that refused passwords produced HTTP 500; the re-review found "two more password leaks". Earlier, on 10 July 2026, a security commit (`fd2aa2e`) had removed a production administrator password that had been in the repository since the initial import (the commit message counts 15 places), and asked for its rotation (OWNER_REVIEW C6).

**A deployment to the wrong cloud project.** On 17 October 2026, according to PR #44, "a bare `gcloud builds submit` ran while the machine's default gcloud project belonged to an unrelated product"; it uploaded the application's source there and created a failing service; "both were removed the same day". The fix added a first build step that refuses any project other than the application's own, and a deploy script that always names the project. The repository does not state who ran the command (OWNER_REVIEW C7).

Common to the four: each was invisible to CI by construction (CI has no credentials, no production, no browser, no deployment), each involved the boundary between the development environment and production, and each was found by someone looking at the running system for another purpose.

# 6 Discussion

## 6.1 Evidence that cannot fail

The common shape of 25 of the 33 coded episodes (classes P, N, S, I and M by primary code), and of the post-October episodes, is verification that was executed — or claimed — but could not have failed for the defect at hand. The agent produced artefacts with every outward sign of rigour: prose naming the method, test files named after the risk

control, negative-control tables, review sections with numbered findings. The failures were rarely careless omissions; they were plausible evidence at the wrong place: the caller instead of the render path, the decision function instead of the request, one file instead of the mounted application, a mock instead of production metadata, jsdom instead of a browser, a re-anchored mutation that no longer removed the control. We read this as the regulated-software form of what Ferris and Bonilla call monitors that measure proxies (Ferris and Bonilla 2026), and as a concrete instance of the inaccurate self-reports that Tang et al. observe at scale (Tang et al. 2026).

## 6.2 What caught it

Three mechanisms recur. *Checks at the locus of effect*: requests through the mounted application, real browsers, round trips through the real store and load paths, enumeration of the routes the application actually mounts, tests run on the pre-fix base. *Mutations that restore the defect as found*, re-run on every integrated tree, with hashes that tie the record to the code. *Adversarial framing by a separate session*: re-verification "briefed to falsify", reviews that reproduce a defect with a failing test before reporting it. None is new; what the case adds is that, with an agent as author, each had to be institutionalised as a rule because the author's own verification repeatedly passed — Davis et al.'s "governance conversion" (Davis et al. 2026) observed in a regulated lifecycle.

Two further channels were not designed as verification at all and found the most severe late defects: browser sessions run to film the product, and the owner's own research studies on the code base. Both look at the running system with a purpose other than confirming that it works. Their yield (19 of 38 records) suggests that, for agent-built software, *use* of the running system by someone with a different goal is a detection channel to plan for, not an accident.

## 6.3 Mutation testing as a negative control: necessary, not sufficient

At release scale the negative-control regime is expensive — 873 hand-written mutations, each run in a throwaway worktree — and it works in the sense the regime intends: a control that stops working makes a test fail, and the record says which. Its limits are those of any mutation analysis (Papadakis et al. 2019; Barr et al. 2015), made sharper by the agent: mutations at the wrong locus pass, mutations drift when code moves, and records that are committed only after a clean run hide their own history. Three practices follow from the case: include the defect as found, byte for byte; bind each control to a behaviour-level test at the locus of effect; and commit every run, including those with unexpected survivors, so that the record shows what the process found, not only where it ended. A pooled detection rate should never be reported as effectiveness.

## 6.4 Review by agent sessions

The 29 review rounds were productive — two blockers and 23 majors in ten days — and some of their findings are exactly what a human reviewer would hope to catch

(DIS on unconfirmed labels; a confirmation guard reading the wrong view). They are also not independent in the regulatory sense: the reviewers are sessions of the same agent family, briefed by the coordinating session, and their reports exist only as the author's summaries. The first integration review of PR #69 found nothing blocking on the commit on which a browser found a clinical-safety defect. Agent review is a cheap, high-yield filter; it is not a substitute for the independent human review that ISO 13485 §7.3.5 and the project's own plan require.

## 6.5 Approval provenance and the semantics of records

The case separates two properties that standards treat as one. *Attribution of the act* — who approved — was much improved by moving the approval out of the agent's session into a verified commit made through GitHub's web interface; it becomes technical only when the agent no longer holds the owner's credentials. *Ownership of the content* — what was approved — was not achieved at all: the approval commit replaced a string the agent had placed, and in at least 15 places that string was part of a sentence whose meaning changed, including the procedure that governs approvals. An approval mechanism for agent-written records therefore needs both a provenance check (who made the commit, with which key) and a semantic check (that each replaced token is an approval field and nothing else) — for example, structured approval fields instead of free-text placeholders, and a CI check that refuses a placeholder outside a field. The permission classifier that refused to let the agent write approval lines was the first effective barrier, but it belongs to the tool vendor (Anthropic 2026b,a), not to the quality system.

## 6.6 Implications

**For standards and notified bodies.** (1) Declare an AI coding agent as a development tool (IEC 62304 §5.1.4) and state what it may not do. (2) For Class C risk controls, require evidence that can fail: a named test at the locus of effect, a recorded negative control including the defect as found, and the full run history. (3) Treat agent-drafted records as claims: verify quotations, counts, statuses and histories mechanically. (4) Bind approvals to people *and* to structured fields; an approval written by the agent into a document is a transcription. (5) Review sessions of the same agent are not independent reviewers.

**For practitioners.** Separate the agent's identity from the owner's (own account, no merge rights, no production credentials); protect the main branch and make CI a gate; isolate every test run from cloud credentials by default; run the product in a real browser before each release; and plan for the records to lag the history.

**For tool builders.** Permission systems that refuse to write approvals or to act on production are valuable, but auditors need them to be configurable, logged and inspectable as part of a quality system.

**For researchers.** The case suggests measurable constructs — escape and persistence of verification failures, locus of a check, drift and survivorship in mutation records, provenance and semantic integrity of approvals, staleness of record statuses — that could be compared across agent-built and human-built regulated projects.

# 7 Threats to validity

We follow the usual classification for case studies (Runeson and Höst 2009; Wohlin et al. 2012; Ralph et al. 2021).

**Construct validity.** The agent's trailer is a self-reported lower bound on its involvement. Most textual evidence (records, notes, PR descriptions) was written by the agent and can over- or understate; we found false statements in the records themselves (class R), outdated statuses (25 of 38) and approval text whose meaning changed (§5.5). We therefore lean on machine evidence (git ancestry, blame, committer, signature verification, record files produced by scripts, CI results) wherever possible and mark agent-reported counts (e.g. test totals) as such. Severity labels in review findings are the agent's own; 88 of 189 findings carry none. "Independent" reviewers are agent sessions. The detection channel of a record is coded from the record's own Source field.

**Internal validity.** Only detected failures are visible; undetected failures are, by construction, missing. The escaped/contained contrast is right-censored and confounded with model changes (eight model strings) and with the introduction of pre-merge review. The rise in findings and records in late October coincides with more review, more browser sessions and more parallel sessions; it does not show that the software became worse or better. The sign-off substitution classification is heuristic (a lower bound on non-placeholder replacements).

**External validity.** One product, one owner, one agent family, one regulatory frame, eleven months, an owner with research interests in the product. We claim analytical, not statistical, generalisation (Flyvbjerg 2006; Yin 2018). The project is unusually record-heavy; projects with fewer records would expose fewer failures to this method, not necessarily fewer failures.

**Reliability and researcher bias.** The author owns the product, operated the agent and approved the changes studied; he is also the subject. The agent under study wrote the analysis scripts, coded the episodes, the review findings and the record channels, and drafted this manuscript. Mitigations: public repository and pinned commits; scripts that re-derive every number; verbatim checks of every coded quotation (81 in the October coding, 208 in the review coding, 38 in the channel coding, and 33 statements quoted from records and pull requests); a script that checks each number of this manuscript against the re-derived data (`NUMBERS_CHECK.md`); a protocol committed before the October analysis; an explicit list of post-hoc decisions (§4.7). **Not yet mitigated:** an independent human second coder with inter-rater agreement (Cohen 1960) on the episodes, review findings and channels, and an independent re-run of the scripts (OWNER_REVIEW B1, B2).

# 8 Ethics

The study involves no human participants other than the author, who is also the owner of the product and consents to the publication of its records. No patient data, no production data and no personal data of third parties were accessed or are reported; the patient-registry incident is described from the code and the records only, and whether any personal data were exposed in production is unknown and is the subject of an

owner action. Names and identifiers of an unrelated product of the owner are withheld. Security weaknesses are reported only after their fixes were merged; the author must confirm that the fixes are deployed and that the exposed administrator password has been rotated before submission (OWNER_REVIEW C4–C6). The agent's mistakes are reported in full; reporting them is the purpose of the study, not a judgement of any person. No ethics-board approval was sought, because the study analyses the author's own engineering records (OWNER_REVIEW A4: confirm against the target journal's policy).

# 9 Conclusion

In one IEC 62304 Class C project in which an AI coding agent wrote nearly all code and lifecycle records, the recurrent failure was not missing verification but verification that could not fail. It was caught by checks that observe the running system, by mutations that restore the defect as found, by review sessions briefed to refute, and — for the most severe late defects — by people and sessions using the product for something else. Mutation-based negative controls scaled to 873 mutations per release and still failed in three identifiable ways. The approval boundary became largely attributable when the owner's act moved out of the agent's session, but not fully while the agent holds the owner's credentials, and it did not make the agent's words the owner's. That is where a regulator, an auditor or a notified body should look first.

# Declarations

**AI-assistance disclosure.** This study is about an AI coding agent, and the same agent family assisted in producing it. Claude Code with Claude models (trailers name eight model strings; the present text was drafted by Claude Opus 5.5) wrote most of the code of MSTool-AI and most of its lifecycle records, including the CAPA and problem records, the integration notes, the negative-control scripts and the review reports analysed here. The same agent wrote the analysis scripts, coded the failure episodes, review findings and record channels (single coder), produced the figures, verified the references, and drafted this manuscript and its supplementary files at the author's request. The author directed the work, made the decisions recorded as his, and is responsible for the content. The agent is not an author. *(The author must review the full text, resolve OWNER_REVIEW.md and align this statement with the target journal's policy on AI-assisted writing — OWNER_REVIEW A3.)*

**Funding.** None declared *(to be confirmed by the author, OWNER_REVIEW A2).*

**Conflicts of interest.** The author owns the product studied (MSTool-AI, software intended to become a commercial medical device), operated the agent and approved the changes studied, so the study evaluates his own work. He is also affiliated with Daita AI *(owner to confirm: his role there, and whether Daita AI has any financial interest in MSTool-AI or in this study).* The agent studied, Claude Code, is a commercial product of Anthropic; the author uses it as a customer and has no other relationship with Anthropic *(owner to confirm). (Ownership or equity, patents, paid roles, and relationships with authors of cited works, to be declared by the author — OWNER_REVIEW A2.)*

**Ethics approval and consent to participate.** No ethics-board approval was sought; the only human participant is the author, who consents (Section 8) *(to be confirmed against the journal's policy — OWNER_ REVIEW A4).*

**Consent for publication.** The author consents to the publication of the project records quoted here; no other person's data are reported (Section 8).

**Data and code availability.** The repository is public. Analysis scripts, coding sheets and the redacted GitHub snapshot: `research/paper_h/`; figures and their script: `docs/research/paper-h/manuscript/figures/`; pinned commits `6209131` and `8805882`. *(To be archived with a DOI before submission, OWNER_ REVIEW B4.)*

**Materials availability.** Not applicable.

**Author contributions (CRediT).** Nicolás Bonilla Vargas: conceptualisation, supervision, resources, validation, writing — review and editing *(to be confirmed after the author's review).*

# References


Alemzadeh H, Iyer RK, Kalbarczyk Z, et al (2013) Analysis of Safety-Critical Computer Failures in Medical Devices. IEEE Security & Privacy 11(4):14–26. https://doi.org/10.1109/MSP.2013.49

Anthropic (2026a) Choose a permission mode – Claude Code Docs. Official product documentation, URL https://code.claude.com/docs/en/permission-modes, accessed 31 October 2026; documents modes default (Manual), acceptEdits, plan, auto, dontAsk, bypassPermissions

Anthropic (2026b) Configure permissions – Claude Code Docs. Official product documentation, URL https://code.claude.com/docs/en/permissions, accessed 31 October 2026

Barr ET, Harman M, McMinn P, et al (2015) The Oracle Problem in Software Testing: A Survey. IEEE Transactions on Software Engineering 41(5):507–525. https://doi.org/10.1109/TSE.2014.2372785

Becker J, Rush N, Barnes E, et al (2025) Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. arXiv preprint arXiv:2507.09089, URL https://arxiv.org/abs/2507.09089, arXiv:2507.09089

Chen M, Tworek J, Jun H, et al (2021) Evaluating Large Language Models Trained on Code. arXiv preprint arXiv:2107.03374, URL https://arxiv.org/abs/2107.03374, arXiv:2107.03374

Cohen J (1960) A Coefficient of Agreement for Nominal Scales. Educational and Psychological Measurement 20(1):37–46. https://doi.org/10.1177/001316446002000104

Davis JC, Amusuo PC, Singla T, et al (2026) Cheap Code, Costly Judgment: A Case Study on Governable Agentic Software Engineering. arXiv preprint

arXiv:2607.01087, URL https://arxiv.org/abs/2607.01087, arXiv:2607.01087

DeMillo RA, Lipton RJ, Sayward FG (1978) Hints on Test Data Selection: Help for the Practicing Programmer. Computer 11(4):34–41. https://doi.org/10.1109/C-M.1978.218136

Easterbrook S, Singer J, Storey MA, et al (2008) Selecting Empirical Methods for Software Engineering Research. In: Guide to Advanced Empirical Software Engineering. Springer, London, p 285–311, https://doi.org/10.1007/978-1-84800-044-5_11

Ehsani R, Pathak S, Rawal S, et al (2026) Where Do AI Coding Agents Fail? An Empirical Study of Failed Agentic Pull Requests in GitHub. In: Proceedings of the 23rd International Conference on Mining Software Repositories (MSR 2026). ACM, pp 807–811, https://doi.org/10.1145/3793302.3793579, arXiv:2601.15195

European Parliament and Council of the European Union (2017) Regulation (EU) 2017/745 of the European Parliament and of the Council of 5 April 2017 on medical devices, amending Directive 2001/83/EC, Regulation (EC) No 178/2002 and Regulation (EC) No 1223/2009 and repealing Council Directives 90/385/EEC and 93/42/EEC. Official Journal of the European Union, L 117, 5 May 2017, URL https://eur-lex.europa.eu/eli/reg/2017/745/oj/eng, medical Device Regulation (MDR); CELEX 32017R0745. Consolidated versions exist up to 2026; state which one is cited

European Parliament and Council of the European Union (2024) Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Official Journal of the European Union, OJ L, 2024/1689, 12 July 2024, URL https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng, cELEX 32024R1689; a consolidated version dated 2026-07-27 exists on EUR-Lex

Fan A, Gokkaya B, Harman M, et al (2023) Large Language Models for Software Engineering: Survey and Open Problems. In: 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE). IEEE, pp 31–53, https://doi.org/10.1109/ICSE-FoSE59343.2023.00008

Ferris L, Bonilla S (2026) A Case Study in Assuring AI-Written Software. arXiv preprint arXiv:2610.08651, URL https://arxiv.org/abs/2610.08651, accepted to the NeurIPS 2026 Meta-Agents Workshop (arXiv comment), arXiv:2610.08651

Flyvbjerg B (2006) Five Misunderstandings About Case-Study Research. Qualitative Inquiry 12(2):219–245. https://doi.org/10.1177/1077800405284363

Fu K (2011) Trustworthy Medical Device Software. In: Public Health Effectiveness of the FDA 510(k) Clearance Process: Measuring Postmarket Performance and Other Select Topics: Workshop Report. The National Academies Press, Washington, DC, p 97–118, https://doi.org/10.17226/13020, appendix D

Gangopadhyay T, Adler R, Liggesmeyer P, et al (2026) From Safety Documentation to Safety Knowledge Support: An Evidence-Grounded LLM Framework for Medical Devices. arXiv preprint arXiv:2608.12025, URL https://arxiv.org/abs/2608.12025, iSSRE 2026 workshop AISQ (arXiv comment); no proceedings DOI found on 2026-10-31, arXiv:2608.12025

GitHub (2026) About commit signature verification – GitHub Docs. Official product documentation, URL https://docs.github.com/en/authentication/managing-commit-signature-verification/about-commit-signature-verification, accessed 31 October 2026; describes automatic GPG signing of web-interface commits with GitHub's web-flow key and the Verified status

Granlund T, Stirbu V, Mikkonen T (2021) Towards Regulatory-Compliant MLOps: Oravizio's Journey from a Machine Learning Experiment to a Deployed Certified Medical Product. SN Computer Science 2(5):342. https://doi.org/10.1007/s42979-021-00726-1

Heeager LT, Nielsen PA (2018) A conceptual model of agile software development in a safety-critical context: A systematic literature review. Information and Software Technology 103:22–39. https://doi.org/10.1016/j.infsof.2018.06.004

Hou X, Zhao Y, Liu Y, et al (2024) Large Language Models for Software Engineering: A Systematic Literature Review. ACM Transactions on Software Engineering and Methodology 33(8):1–79. https://doi.org/10.1145/3695988

IMDRF Software as a Medical Device Working Group (2014) "Software as a Medical Device": Possible Framework for Risk Categorization and Corresponding Considerations. IMDRF/SaMD WG/N12FINAL:2014, International Medical Device Regulators Forum, URL https://www.imdrf.org/documents/software-medical-device-possible-framework-risk-categorization-and-corresponding-considerations, final document, 18 September 2014

International Electrotechnical Commission (2015) IEC 62304:2006+AMD1:2015 CSV: Medical device software – software life cycle processes. International standard, consolidated version (edition 1.1), URL https://webstore.iec.ch/en/publication/22794, first edition (2006) consolidated with Amendment 1 (2015); publication date 2015-06-26

International Electrotechnical Commission (2016) IEC 82304-1:2016: Health software – part 1: General requirements for product safety. International standard, edition 1.0, URL https://webstore.iec.ch/en/publication/26120, publication date 2016-10-27

International Organization for Standardization (2016) ISO 13485:2016: Medical devices – quality management systems – requirements for regulatory purposes. International standard, 3rd edition, URL https://www.iso.org/standard/59752.html

International Organization for Standardization (2019) ISO 14971:2019: Medical devices – application of risk management to medical devices. International standard, 3rd edition, URL https://www.iso.org/standard/72704.html

Jia Y, Harman M (2011) An Analysis and Survey of the Development of Mutation Testing. IEEE Transactions on Software Engineering 37(5):649–678. https://doi.org/10.1109/TSE.2010.62

Jimenez CE, Yang J, Wettig A, et al (2024) SWE-bench: Can language models resolve real-world GitHub issues? In: The Twelfth International Conference on Learning Representations (ICLR 2024), URL https://openreview.net/forum?id=VTF8yNQM66, arXiv:2310.06770

Just R, Jalali D, Inozemtseva L, et al (2014) Are Mutants a Valid Substitute for Real Faults in Software Testing? In: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2014). ACM, pp 654–665, https://doi.org/10.1145/2635868.2635929

Kirchner S, Knoll AC (2025) Generating Automotive Code: Large Language Models for Software Development and Verification in Safety-Critical Systems. In: 2025 IEEE Intelligent Vehicles Symposium (IV). IEEE, pp 813–820, https://doi.org/10.1109/IV64158.2025.11097503, arXiv:2506.04038

Laukkarinen T, Kuusinen K, Mikkonen T (2017) DevOps in regulated software development: Case medical devices. In: 2017 IEEE/ACM 39th International Conference on Software Engineering: New Ideas and Emerging Technologies Results Track (ICSE-NIER). IEEE, pp 15–18, https://doi.org/10.1109/ICSE-NIER.2017.20

Laukkarinen T, Kuusinen K, Mikkonen T (2018) Regulated software meets DevOps. Information and Software Technology 97:176–178. https://doi.org/10.1016/j.infsof.2018.01.011

Leveson NG, Turner CS (1993) An Investigation of the Therac-25 Accidents. Computer 26(7):18–41. https://doi.org/10.1109/MC.1993.274940

Li H, Zhang H, Hassan AE (2025) The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. arXiv preprint arXiv:2507.15003, URL https://arxiv.org/abs/2507.15003, introduces the AIDev dataset, arXiv:2507.15003

Liu F, Liu Y, Shi L, et al (2026a) Beyond Functional Correctness: Exploring Hallucinations in LLM-Generated Code. IEEE Transactions on Software Engineering 52(3):1037–1055. https://doi.org/10.1109/TSE.2026.3657432, earlier arXiv versions titled "Exploring and Evaluating Hallucinations in LLM-Powered Code Generation", arXiv:2404.00971

Liu J, Wang K, Chen Y, et al (2026b) Large Language Model-Based Agents for Software Engineering: A Survey. ACM Transactions on Software Engineering and Methodology https://doi.org/10.1145/3796507, published online 5 March 2026; volume/issue not yet assigned in Crossref on 2026-10-31, arXiv:2409.02977

Liu M, Wang J, Lin T, et al (2024) An Empirical Study of the Code Generation of Safety-Critical Software Using LLMs. Applied Sciences 14(3):1046. https://doi.org/10.3390/app14031046

Madeyski L, Orzeszyna W, Torkar R, et al (2014) Overcoming the Equivalent Mutant Problem: A Systematic Literature Review and a Comparative Experiment of Second Order Mutation. IEEE Transactions on Software Engineering 40(1):23–42. https://doi.org/10.1109/TSE.2013.44

Maharjan S (2026) Trust but Trace: AI Coding Assistants in Software Developed under Regulatory Design Controls. SSRN preprint, https://doi.org/10.2139/ssrn.7349799

McHugh M, McCaffery F, Casey V (2012) Barriers to Adopting Agile Practices When Developing Medical Device Software. In: Software Process Improvement and Capability Determination (SPICE 2012). Springer, Berlin, Heidelberg, Communications in Computer and Information Science, pp 141–147, https://doi.org/10.1007/978-3-642-30439-2_13

Medical Device Coordination Group (2025) MDCG 2019-11 rev.1: Guidance on qualification and classification of software in Regulation (EU) 2017/745 – MDR and Regulation (EU) 2017/746 – IVDR. MDCG endorsed guidance document, European Commission, URL https://health.ec.europa.eu/document/download/b45335c5-1679-4c71-a91c-fc7a4d37f12b_en?filename=mdcg_2019_11_en.pdf, original version October 2019; revision 1 June 2025

Nouri A, Andersson J, De Jesus Hornig K, et al (2025) On Simulation-Guided LLM-based Code Generation for Safe Autonomous Driving Software. In: Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering (EASE 2025). ACM, pp 1097–1106, https://doi.org/10.1145/3756681.3756987, arXiv:2504.02141

Papadakis M, Shin D, Yoo S, et al (2018) Are Mutation Scores Correlated with Real Fault Detection? In: Proceedings of the 40th International Conference on Software Engineering (ICSE 2018). ACM, pp 537–548, https://doi.org/10.1145/3180155.3180183

Papadakis M, Kintis M, Zhang JM, et al (2019) Mutation Testing Advances: An Analysis and Survey. In: Advances in Computers, vol 112. Elsevier, p 275–378, https://doi.org/10.1016/bs.adcom.2018.03.015

Pearce H, Ahmad B, Tan B, et al (2022) Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. In: 2022 IEEE Symposium on

Security and Privacy (SP). IEEE, pp 754–768, https://doi.org/10.1109/SP46214.2022.9833571

Perry N, Srivastava M, Kumar D, et al (2023) Do Users Write More Insecure Code with AI Assistants? In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23). ACM, pp 2785–2799, https://doi.org/10.1145/3576915.3623157

Petrović G, Ivanković M (2018) State of Mutation Testing at Google. In: Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP 2018). ACM, pp 163–171, https://doi.org/10.1145/3183519.3183521

Petrović G, Ivanković M, Fraser G, et al (2022) Practical Mutation Testing at Scale: A View from Google. IEEE Transactions on Software Engineering 48(10):3900–3912. https://doi.org/10.1109/TSE.2021.3107634

Ralph P, bin Ali N, Baltes S, et al (2021) Empirical Standards for Software Engineering Research. arXiv preprint arXiv:2010.03525 (v2), URL https://arxiv.org/abs/2010.03525, aCM SIGSOFT Empirical Standards; v1 7 Oct 2020, v2 4 Mar 2021; living version at https://github.com/acmsigsoft/EmpiricalStandards, arXiv:2010.03525

Runeson P, Höst M (2009) Guidelines for conducting and reporting case study research in software engineering. Empirical Software Engineering 14(2):131–164. https://doi.org/10.1007/s10664-008-9102-8

Runeson P, Höst M, Rainer A, et al (2012) Case Study Research in Software Engineering: Guidelines and Examples. Wiley, Hoboken, NJ, https://doi.org/10.1002/9781118181034

Spracklen J, Wijewickrama R, Sakib AHMN, et al (2025) We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs. In: 34th USENIX Security Symposium (USENIX Security 25). USENIX Association, Seattle, WA, pp 3687–3706, URL https://www.usenix.org/conference/usenixsecurity25/presentation/spracklen, arXiv:2406.10279

Stirbu V, Granlund T, Mikkonen T (2023) Continuous design control for machine learning in certified medical systems. Software Quality Journal 31(2):307–333. https://doi.org/10.1007/s11219-022-09601-5, published online 17 October 2022

Sun S, Staron M (2026) Agentic Pipelines in Embedded Software Engineering: Emerging Practices and Challenges. In: 2026 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER). IEEE, pp 136–146, https://doi.org/10.1109/SANER67736.2026.00024, arXiv:2601.10220

Tang N, Chen C, Xu G, et al (2026) How Coding Agents Fail Their Users: A Large-Scale Analysis of Developer-Agent Misalignment in 20,574 Real-World Sessions. In:

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing (EMNLP 2026). Association for Computational Linguistics, pp 23264–23282, https://doi.org/10.18653/2026.emnlp-main.1011, arXiv:2605.29442

U.S. Food and Drug Administration (2002) General Principles of Software Validation; Final Guidance for Industry and FDA Staff. Final guidance, CDRH and CBER, docket FDA-1997-D-0029, URL https://www.fda.gov/regulatory-information/search-fda-guidance-documents/general-principles-software-validation

U.S. Food and Drug Administration (2023) Content of Premarket Submissions for Device Software Functions: Guidance for Industry and Food and Drug Administration Staff. Final guidance, docket FDA-2021-D-0775, URL https://www.fda.gov/regulatory-information/search-fda-guidance-documents/content-premarket-submissions-device-software-functions, issued June 2023 (page current as of 2023-06-14)

U.S. Food and Drug Administration (2024) Marketing Submission Recommendations for a Predetermined Change Control Plan for Artificial Intelligence-Enabled Device Software Functions. Final guidance, docket FDA-2022-D-2628, URL https://www.fda.gov/regulatory-information/search-fda-guidance-documents/marketing-submission-recommendations-predetermined-change-control-plan-artificial-intelligence, final guidance issued 4 December 2024; the version now posted is dated August 2025 (page current as of 2025-08-18)

U.S. Food and Drug Administration (2025) Artificial Intelligence-Enabled Device Software Functions: Lifecycle Management and Marketing Submission Recommendations. Draft guidance, docket FDA-2024-D-4488, URL https://www.fda.gov/regulatory-information/search-fda-guidance-documents/artificial-intelligence-enabled-device-software-functions-lifecycle-management-and-marketing, draft (not for implementation); issued January 2025 (page current as of 2025-01-07); still draft on 2026-10-31

U.S. Food and Drug Administration (2026) Computer Software Assurance for Production and Quality Management System Software: Guidance for Industry and Food and Drug Administration Staff. Final guidance, CDRH and CBER, docket FDA-2022-D-0795, URL https://www.fda.gov/regulatory-information/search-fda-guidance-documents/computer-software-assurance-production-and-quality-management-system-software, supersedes the final guidance "Computer Software Assurance for Production and Quality System Software" issued 24 September 2025 (draft September 2022)

Wallace DR, Kuhn DR (2001) Failure Modes in Medical Device Software: An Analysis of 15 Years of Recall Data. International Journal of Reliability, Quality and Safety Engineering 8(4):351–371. https://doi.org/10.1142/S021853930100058X

Wang T, Beaudry J, Damato AL, et al (2026) AI-assisted software development for brachytherapy seed inventory management. Journal of Applied Clinical Medical Physics 27(9):e70795. https://doi.org/10.1002/acm2.70795

Wang X, Li B, Song Y, et al (2025) OpenHands: An open platform for AI software developers as generalist agents. In: The Thirteenth International Conference on Learning Representations (ICLR 2025), URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/a4b6ad6b48850c0c331d1259fc66a69c-Abstract-Conference.html, arXiv:2407.16741

Watanabe M, Li H, Kashiwa Y, et al (2026) On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub. ACM Transactions on Software Engineering and Methodology 35(10):1–24. https://doi.org/10.1145/3798166, arXiv:2509.14745

Witztum A, Wang Y, Wang D (2026) AI-coded radiation therapy software requires unique quality assurance. Journal of Applied Clinical Medical Physics 27(10):e70848. https://doi.org/10.1002/acm2.70848

Wohlin C, Runeson P, Höst M, et al (2012) Experimentation in Software Engineering. Springer, Berlin, Heidelberg, https://doi.org/10.1007/978-3-642-29044-2

Yang J, Jimenez CE, Wettig A, et al (2024) SWE-agent: Agent-computer interfaces enable automated software engineering. In: Advances in Neural Information Processing Systems 37 (NeurIPS 2024), pp 50528–50652, https://doi.org/10.52202/079017-1601, arXiv:2405.15793

Yin RK (2018) Case Study Research and Applications: Design and Methods, 6th edn. SAGE, Los Angeles, CA

# A Reproduction

```
# from the repository root, with git and the pinned commits available (all on origin)
python research/paper_h/run_all.py --check                # October analysis (main @ 6209131)
python research/paper_h/manuscript_data.py --check        # this update (main @ 8805882), ~5 min
python docs/research/paper-h/manuscript/figures/make_figures.py
python -m pytest research/paper_h/tests -q                # parsers and headline re-derivations
python research/paper_h/snapshot_github_full.py           # optional: a newer GitHub snapshot
↪  (needs gh)
```

Every number in this manuscript is listed with its source key in `NUMBERS_CHECK.md`.

# B Glossary of project terms

- **CAPA** — corrective and preventive action record (QP-002). **PRB** — software problem report (SPR-001, IEC 62304 §9). **RC** — risk control (RMF-001, ISO 14971). **Manifest** — the machine-checked binding of each RC to tests and a negative-control result.
- **Negative control** — a deliberate mutation that removes or neutralises a risk control; the bound tests must fail. **Declared equivalent** — a mutation declared in advance, with a reason, as unable to change observable behaviour. **Tests on base** — tests written first, run on the pre-fix commit.

- **Escaped / contained** — whether defective evidence reached `main` before it was detected.
- **Sign-off commit** — a commit by the owner that replaces `PENDING owner approval` with an approval line for one pull request.